One content platform on GCP

A decision framework for replacing 27 Drupal sites with one governed content platform

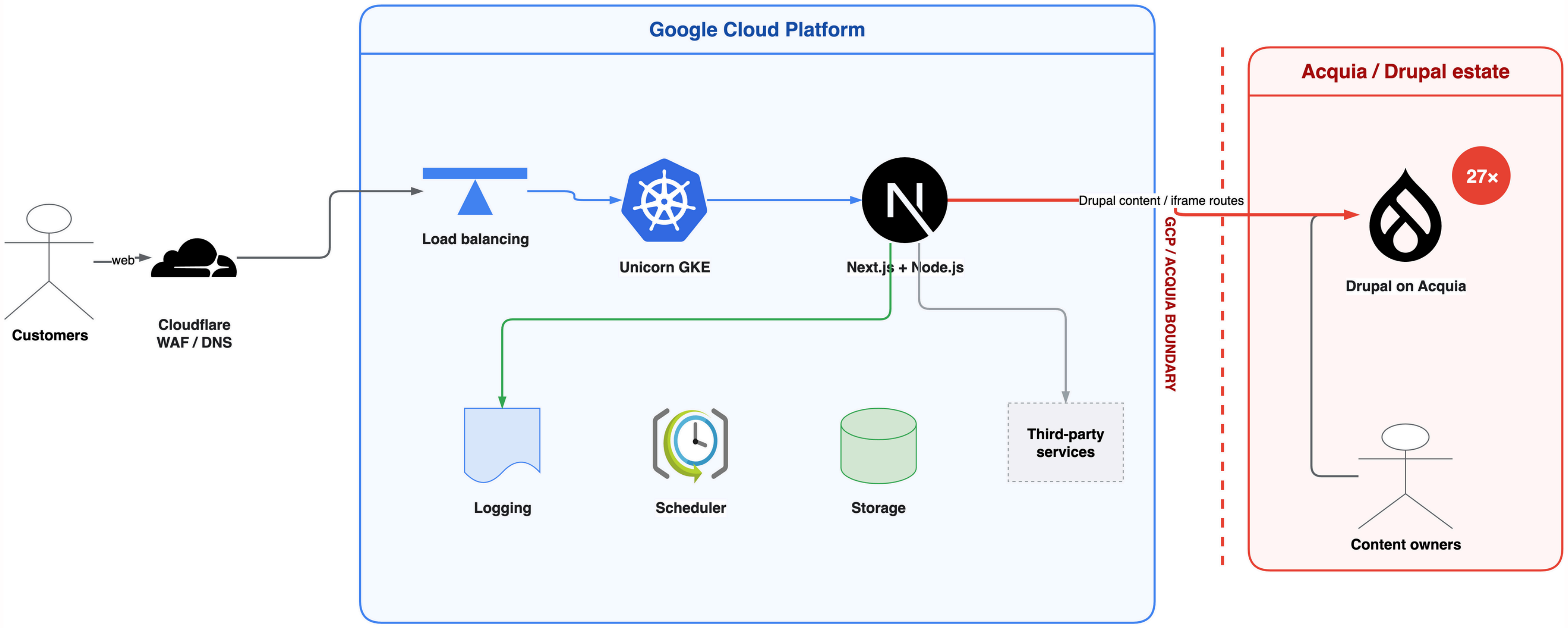
DECISION PROPOSAL · 30 AUGUST 2026

WHY CHANGE · CURRENT ARCHITECTURE

Today, Next.js already serves a mix of native pages and Drupal content

Current platform context: Next.js composes GCP and Drupal delivery

CMS delivery only: native Next.js routes and legacy Drupal / iframe routes cross the GCP / Acquia boundary directly.



Mixed delivery exists today; migration can move route ownership gradually instead of waiting for one big cutover.

WHY CHANGE · OPERATING MODEL

Replace 27 separate CMS stacks with one governed platform

Current estate

27×

Runtime	26 country instances + 1 global instance
Change	Repeated patches, releases, and testing
Cost	Acquia + Drupal + Varnish + cross-cloud work
Data	Reuse and reporting across isolated databases

CENTRALIZE GOVERNANCE

→

Target platform

1

Runtime	One logical CMS on GCP
Change	Shared deployment and regression controls
Cost	GCP + support + engineering
Data	Country policy + shared global content

The business case comes from retiring repeated platform work across 27 stacks.

DECISION FRAME · REQUIRED OUTCOMES

Improve publishing without weakening engineering control

01

Platform ownership

GCP, PostgreSQL, GCS, backup, and restore.

02

Global publishing

26 country sites, one global site, seven languages, and scoped access.

03

Editorial control

Preview, review, approval, scheduling, and translation state.

04

Structured experience

Approved blocks and rich text, not arbitrary styling.

05

Digital operations

Forms, SEO, audit, observability, and environment promotion.

06

Productivity + transition

Enterprise AI, support, and controlled Drupal coexistence.

These six outcomes define the minimum acceptable platform.

Both candidates meet the baseline; the operating model decides

Shared enterprise baseline

Self-hosting on GCP, PostgreSQL, GCS, headless APIs, country and language controls, audit history, revisions, and Enterprise AI.

Payload advantage

React and TypeScript alignment, packaged publishing workflows, static headless A/B testing, official headless forms, and an MIT-licensed core.

Directus advantage

Packaged Environment Sync, mature Studio automation and Flows, broad data-platform flexibility, and existing internal familiarity.

Evidence still required

Enterprise feature scope, tenancy isolation, upgrades, support and SLA, commercial terms, and planning-horizon total cost.

> **Both are viable. The choice is how much adaptation ONE accepts across content, engineering, and ownership.**

CANDIDATE COMPARISON · VENDOR CONFIDENCE

Both vendors are credible; they optimize for different models

Payload

Application engineering platform

- MIT-licensed core and self-hosting
- React, TypeScript, and Next.js alignment
- Enterprise support and global customer references
- Figma backing with continued open-source commitment

Directus

Data and content operations platform

- Two-decade product history
- Large developer and extension ecosystem
- Formal support and global customer references
- Strong Studio, Flows, and Environment Sync

Credibility is not the deciding factor

Both vendors have enterprise products, communities, customers, and formal support paths.

Fit is the deciding factor

ONE needs the platform that best supports its publishing model and existing engineering organization.

> Vendor-documented capabilities remain subject to demonstration and contract confirmation.

VENDOR DECISION · RECOMMENDATION

Recommend Payload for the strongest fit across content, engineering, and ownership

Both platforms meet the enterprise CMS baseline. Payload requires less adaptation in the areas that define ONE's long-term operating model.

Better content operations

Enterprise publishing workflows, static A/B testing, AI-assisted writing, translation and image generation, plus an official headless Form Builder.

One engineering model

Next.js, React, and TypeScript align the CMS, Admin extensions, design system, tests, migration logic, and daily development.

Long-term control

GCP self-hosting, PostgreSQL, GCS, an MIT-licensed core, activity logs, revisions, and Enterprise support keep the platform governable and replaceable.

Directus remains credible

It leads in packaged Environment Sync and offers mature Studio automation, Flows, AI, and broad data-platform flexibility.

The deciding difference

Payload combines comparable Enterprise authoring capability with a materially stronger fit for how ONE builds and owns digital products.

Directus is a strong option; Payload is the better long-term fit for ONE.

CONTENT VALUE · GLOBAL PUBLISHING

Content teams gain one governed model and a faster daily workflow

27

site scopes

26 country sites + 1 global site

7 supported languages

- English
- Chinese
- Japanese
- Korean
- Spanish
- Portuguese
- French

Write

Create within approved blocks and fields

→

Preview

See the actual Next.js experience while editing

→

Review + translate

Visible status, feedback, and role-based transitions

→

Publish

Approve, schedule, and publish with accountability

Country defines ownership and access. Language is a separate publishing choice.

CONTENT VALUE · PACKAGED CAPABILITIES

Payload packages three capabilities we would otherwise assemble

Publishing workflow

Multi-step approvals, field-level access, notifications, and inline feedback provide the closest documented fit for ONE's target workflow.

Headless forms

The official Form Builder maps editor-defined schemas to accessible Next.js components and typed submission controls.

Governed experimentation

Editors manage approved variants in the CMS while Next.js delivers them statically with ONE's selected analytics.

Payload starting point

Packaged workflow, official headless forms, and static headless variants reduce integration work.

Directus starting point

Strong Studio and Flows; approval stages, headless forms, and experimentation require more composition.

Both require demonstration; Payload is closer to the content operating model ONE wants.

CONTENT VALUE · ENTERPRISE AI

Enterprise AI turns the CMS into a shared productivity platform

Writers + translators

Drafting, rewriting, image generation, and multilingual publishing with human review.

Editors + content operations

Brand prompts, glossaries, structured-content checks, permissions, and reusable governance.

Developers

Approved tools can work with CMS schemas and content through permission-scoped MCP connections.

Payload Enterprise

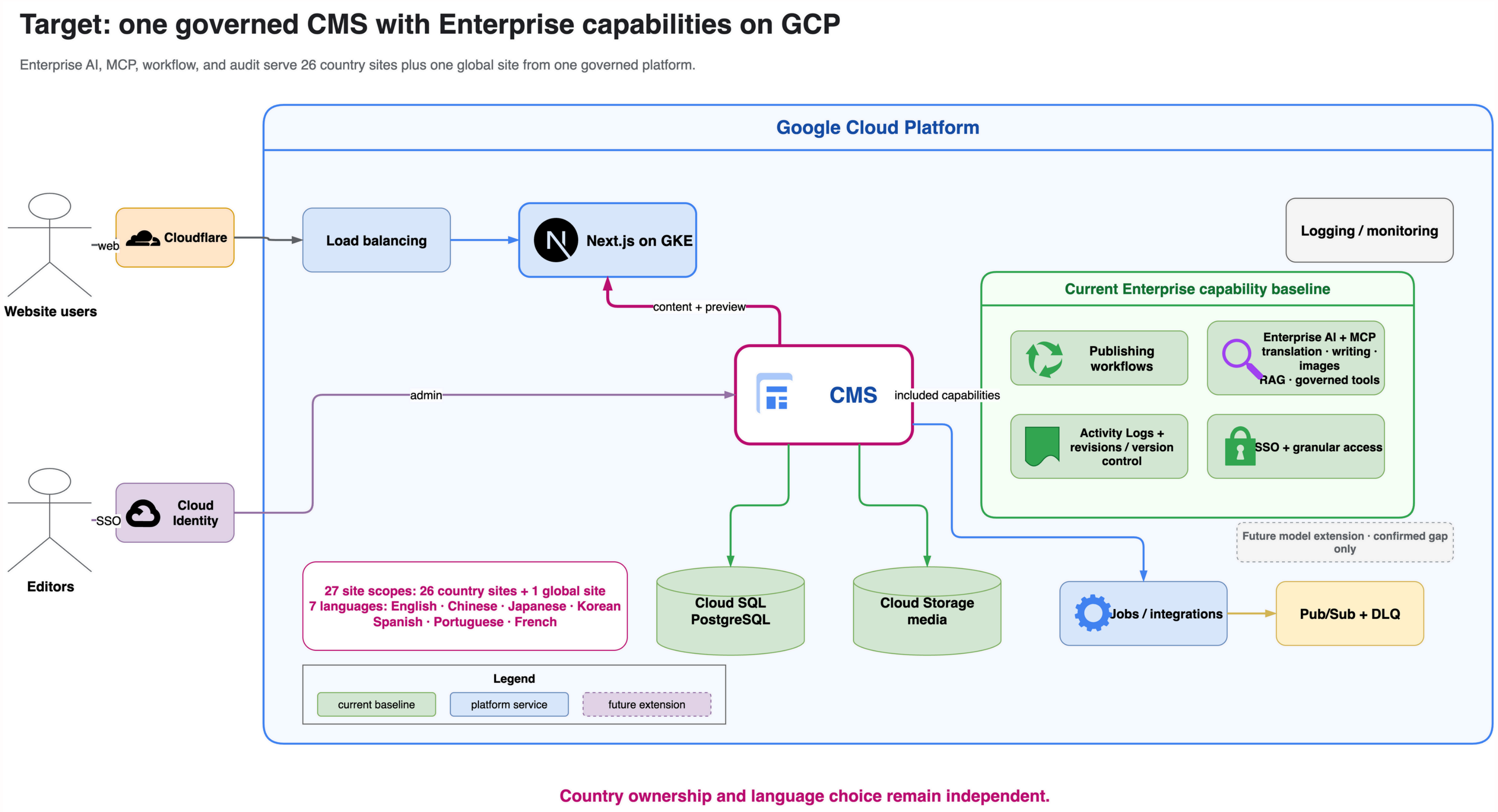
Translation, image generation, writing assistant, permissions, AI content retrieval, and MCP for approved tools.

Directus Enterprise

Studio Assistant, multi-language translation, custom models, and MCP for editors and developers.

Consolidation lets ONE govern AI once across all country and global content operations.

One governed CMS platform can replace 27 separate CMS stacks



> Country and language controls remain independent while the platform, operations, and Enterprise capabilities are shared.

PLATFORM MODEL · ENGINEERING AND OPERATIONS

Payload fits the way ONE builds and operates digital products

Build in one stack

CMS schemas, hooks, jobs, Admin extensions, frontend integration, and tests remain React and TypeScript work.

Govern change

Versioned migrations, controlled reference data, activity logs, revisions, and tenant-aware policy keep changes reviewable.

Operate on GCP

Cloud SQL, GCS, logging, monitoring, backup, restore, security controls, and support fit the existing platform model.

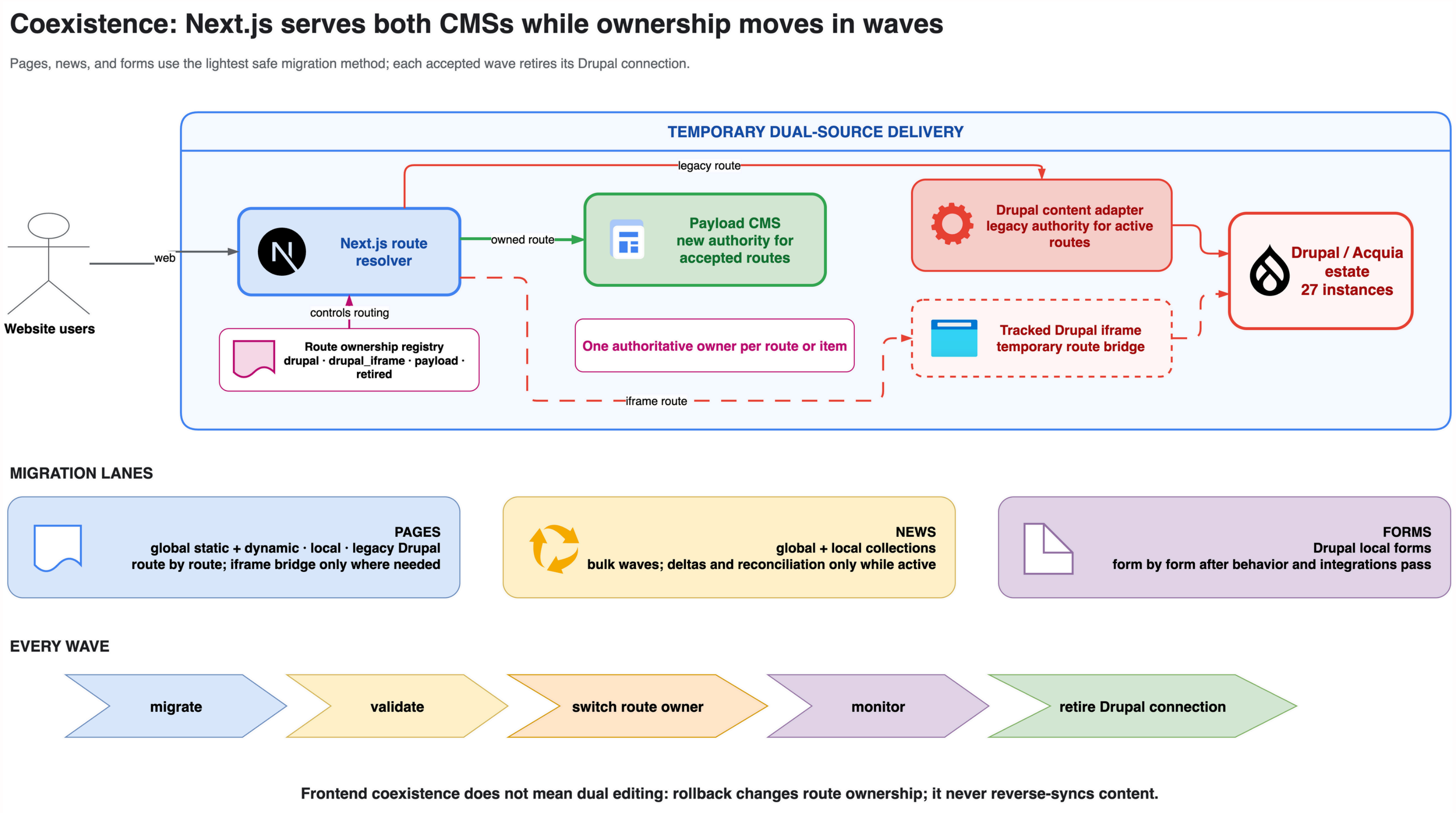
Next.js + React + TypeScript

Payload Admin + APIs + jobs	Shared UI + tests + telemetry
PostgreSQL	GCS on GCP

Directus leads in packaged Environment Sync; Payload leads in one application engineering model.

TRANSITION · CONTROLLED MIGRATION

Migrate in waves while Next.js serves both CMSs



One owner per route or item · route-level rollback · retire each Drupal connection after its wave is accepted

BUSINESS CASE · TOTAL COST

Quantify the business case from retired duplication and total cost

Retire

Repeated operating work

Acquia, Drupal, Varnish, cross-cloud coordination, repeated releases, and duplicated support across 27 stacks.

Count

Target and transition cost

Vendor terms, GCP, implementation, migration, dual-run overlap, support, security, and ongoing engineering.

Measure

Business and delivery value

Publishing cycle time, release effort, incident ownership, reuse, migration progress, and operating cost.

Inputs required

Current contracts and staffing, workload data, migration effort, vendor quotes, and support terms.

Decision output

Verified total cost and value across the approved delivery and operating horizon.

> The case is reduced duplication, not an assumption that the replacement CMS is free.

POC SCOPE · IMPLEMENTATION EVIDENCE

The planned Payload PoC will confirm four implementation gates

01
Content outcomes

Country and language isolation, preview, workflow, forms, experimentation, AI, trash, and restore.

02
Platform operations

GKE, Cloud SQL, GCS, logging, backup, restore, security, and upgrade safety.

03
Migration safety

Dual-source routing, iframe tracking, representative content waves, rollback, and connection retirement.

04
Enterprise + commercial

Feature scope, permissions, audit, SLA, escalation, support, security, and contract terms.

Evidence meets the gates → implementation plan + quantified TCO

Material gap → revisit the decision using the Directus evidence

> The PoC converts remaining assumptions into implementation evidence.

Thank you for listening

Questions and discussion